marinedustrie.de → Skipily: Shopify-koppeling via Make.com

Stapsgewijze handleiding. Met Make-AI: de AI-scenario-assistent bouwt het raamwerk, een AI-tekstmodule verwerkt elk product (naam inkorten, HTML verwijderen, categorie voorstellen). Status: productsync via x-api-key, belastbaar met rate-limit.

---

1. Overzicht (twee richtingen)

- **Producten:** Shopify (marinedustrie.de) → Make → Skipily-shop (automatisch bij aanmaken/wijzigen).
- **Bestellingen:** Skipily → Make → Shopify (bestelling/notitie aanmaken). Optioneel, achteraan beschreven.

2. Wat je nodig hebt

- Skipily-**provider-sleutel** (provider-portaal, Pro-tarief) voor de header x-api-key.
- Shopify-beheer van marinedustrie.de (voor webhooks).
- Een Make.com-account. Voor de AI-tekstmodule een AI-toegang in Make (Make AI of een gekoppeld OpenAI/Claude-account).

3. Stroom A: producten naar Skipily (stap voor stap)

Stap 1: Scenario via Make-AI aanmaken

In Make: **Create a new scenario** → gebruik bovenaan de AI-assistent („*Create with AI*“ / promptveld) en plak de volgende tekst. De AI maakt trigger, AI-module en HTTP-module als raamwerk aan:

AI (Make): Bouw een Make-scenario: Trigger = Custom Webhook (ontvangt Shopify products/create en products/update). Daarna een AI-tekstmodule die uit het Shopify-product de naam inkort, de omschrijving van HTML ontdoet en een hoofdcategorie voorstelt. Tot slot een HTTP-module: POST JSON naar een Skipily-API met header x-api-key.

De AI maakt een concept. De details (URL, header, mapping) stel je in de volgende stappen in.

Stap 2: Trigger = Custom Webhook

- Eerste module **Webhooks > Custom webhook** → **Add** → de aangemaakte **URL kopieren**.

Stap 3: Shopify-webhooks naar de Make-URL

- Shopify-beheer: **Instellingen** → **Meldingen** → **Webhooks**.
- Twee webhooks (formaat **JSON**) naar de Make-URL aanmaken: **Product aangemaakt** (products/create) en **Product bijgewerkt** (products/update).

Let op: Om te testen in Shopify een product opslaan. In Make onderaan „Run once“ aanzetten, zodat de webhook de datastructuur leert.

Stap 4: AI-module - productdata verwerken

Module **AI-tekst** (Make AI / OpenAI / Claude). Ze krijgt het Shopify-product en levert nette JSON (korte naam, omschrijving zonder HTML, passende categorie). Zo wordt uit de vaak lange marinedustrie-titels een duidelijke shopvermelding.

AI (Make): Je krijgt een Shopify-product (title, vendor, body_html, product_type, tags). Geef ALLEEN JSON terug: {"name": korte duidelijke productnaam zonder reclametaal, "description": omschrijving als platte tekst zonder HTML (max 400 tekens), "category": een van de Skipily-categorieen die het best past}. Kies de categorie uit deze lijst: .

Let op: De Skipily-categorienamen haal je eenmalig op: GET .../rest/v1/product_categories?select=name_de,slug (header apikey: anon). Plak die lijst in de prompt. Zonder toewijzing blijft category_id = null (product werkt, alleen zonder categorie).

Stap 5: HTTP-module - naar Skipily sturen

Module **HTTP > Make a request**. Methode POST. In de velden de webhook- en AI-uitvoer mappen (Make stelt toewijzingen met *„Map“* deels via AI voor):

```
POST https://vcjwlyqkfkszumdrfvtm.supabase.co/functions/v1/products-api
Header:  Content-Type: application/json
         x-api-key: <JOUW_SKIPILY_PROVIDER_KEY>

{
  "sku":          "{{1.variants[1].sku}}",
  "name":         "{{ opgeschoonde_naam }}",
  "manufacturer": "{{1.vendor}}",
  "description":  "{{ opgeschoonde_omschrijving }}",
  "price":        {{1.variants[1].price}},
  "currency":     "EUR",
  "ean":          "{{1.variants[1].barcode}}",
  "stock_quantity": {{1.variants[1].inventory_quantity}},
  "in_stock":     {{1.variants[1].inventory_quantity > 0}},
  "weight_kg":    {{1.variants[1].grams / 1000}},
  "images":       ["{{1.image.src}}"],
  "category_id":  {{ skipily_category_id_of_null }},
  "is_active":    true
}
```

Veldtoewijzing Shopify → Skipily:

Shopify-veld	Skipily-veld (products-api)
title	name (via AI ingekort)
vendor	manufacturer
body_html	description (via AI, HTML verwijderd)
variants[].price	price resp. variants[].price
variants[].sku	sku resp. variants[].sku
variants[].barcode	ean
variants[].inventory_quantity	stock_quantity / in_stock
variants[].grams	weight_kg (grams/1000)
options + variants	variant_options + variants (1 regel per variant)
image.src / images	images[] (eerste afbeelding = hoofdafbeelding)
product_type / tags	category_id (via AI-voorstel + toewijzing)

Stap 6: Testen

- In Shopify een product aanmaken/wijzigen → in Make draait het scenario → in de Skipily-shop controleren.
- Verschijnt het product met nette naam/prijs/voorraad, dan klopt het.

Stap 7: Live zetten

- In Make onderaan **Scheduling** op **ON**. Vanaf nu loopt de sync automatisch bij elke Shopify-wijziging.

4. Varianten (diameter, lengte, kleur)

- variant_options = Shopify-opties (naam + waarden). variants = een regel per Shopify-variant met eigen prijs/voorraad/SKU.
- variants vervangt bij POST/PUT ALLE bestaande varianten van een product.

5. Stroom B: bestellingen Skipily -> Shopify (optioneel)

- Tweede scenario met Custom-webhook. In het Skipily-providerprofiel de Make-URL als bestel-webhook opgeven.

- Skipily stuurt bij bestellingen ondertekend daarheen. Maak in Make daaruit een Shopify-bestelling (Admin-API) of notitie/mail.

6. Goed om te weten

- **Nieuw vs. bijwerken:** POST maakt aan, PUT ?id=<uuid> werkt bij. Voor correct bijwerken de Skipily-uuid per Shopify-product bewaren in een Make **Data Store**.
- **Voorraad:** products/update en de Shopify Admin-API leveren inventory_quantity; optioneel extra inventory_levels/update.
- **Rate-limit:** de Skipily-products-api is belastbaar (schrijf-/leeslimiet per provider/IP). Bij massale import in blokken sturen.
- **AI-kosten in de hand:** de AI-module alleen in de productstroom gebruiken (niet in lussen), korte prompts, resultaat in de Data Store cachen zodat onveranderde producten de AI niet opnieuw aanroepen.
- **Vertaling:** Skipily vertaalt naam/omschrijving on-demand in 6 talen. De AI in Make hoeft NIET te vertalen, alleen op te schonen/in te korten.